Programação Avançada

Unidade 2: Estruturas de Dados

Aula 07: Fila e Pilha

Prof. Eduardo Corrêa Gonçalves

06/05/2026

Sumário

Fila

- O que é Fila?
- Criação
- Propriedades e Operações

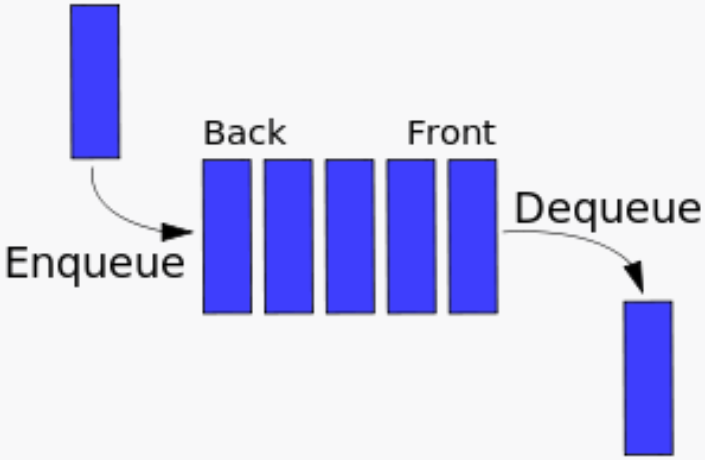
Pilha

- O que é Pilha?
- Criação
- Propriedades e Operações

pacote ‘collections.deque’

Fila (1/9)

- **O que é Fila (*queue*)?**
  - É um container de elementos que são inseridos e removidos de acordo com o conceito **FIFO** (*First-In-First-Out* = primeiro a entrar é o primeiro a sair).
    - É como se fosse uma fila de atendimento. A primeira pessoa a chegar, será a primeira a ser atendida. Quem for chegando tem que entrar no fim da fila.
  - Suporta apenas **duas operações**:
    - *enqueue* (enfileirar) = inserir elemento no final.
    - *dequeue* (desenfileirar) = remover elemento do início.



Fila (2/9)

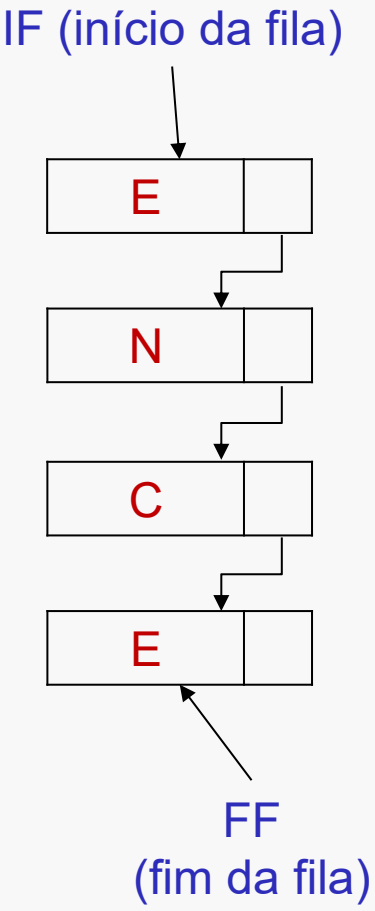
- **Vamos implementar uma Fila em POO com os seguintes métodos:**

Nome	Complexidade	Utilização	Descrição
__init__	O(1)	f = Fila()	Cria a fila
inserir	O(1)	f.inserir(x)	Insere o item *x* no fim da fila
remover	O(1)	x = f.remover()	Remove e retorna o elemento do início da fila
valor_inicio()	O(1)	x = f.valor_inicio()	Retorna o valor do elemento do início da fila sem removê-lo.
vazia	O(1)	f.vazia()	Retorna True se a fila estiver vazia ou False se tiver ao menos um elemento.
tamanho	O(1)	tam = f.tamanho()	Retorna a quantidade de elementos da fila.

- Veja que todas as operações são O(1) !!!
- Muitos algoritmos importantes usam a fila.
  - Um exemplo é o BFS, que veremos na semana que vem que usa fila para determinar ordem de processamento dos nós de um grafo (*na semana que vem implementaremos um grafo que simula uma rede social*)

Fila (3/9)

- **Abordagem básica:**
  - A implementação será feita com base em uma lista encadeada.
  - Precisamos de dois ponteiros:
    - IF – para o início da fila
    - FF – para o fim da fila
  - Cada elemento terá um ponteiro para o elemento que estiver logo atrás dele na fila.
    - Esse ponteiro é chamado de **elo**.
  - *Também é possível implementar uma fila por array, mas não veremos em nosso curso.*



Fila (4/9)

- É preciso definir duas classes:

  - **No**: representação de um nó da fila (dados + elo)

```
class No:
    def __init__(self, dados):
        self.info = dados
        self.elo = None
```

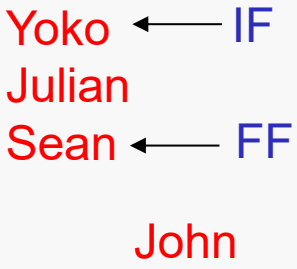
  - **Fila**: p/ representar a fila propriamente dita (nós enfileirados)

```
class Fila:

    def __init__(self):
        #ponteiro para o início da fila
        self.IF = None

        #ponteiro para o fim da fila
        self.FF = None

        #num. de elementos
        self.N = 0
```

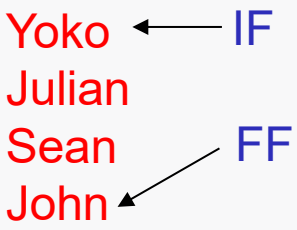
Fila (5/9)

- **Operação inserir (enfileirar)**
  - Esta é a fila original “f”:

  - Criamos um novo nó ...:



  - E aí, basta fazer:
    - O elo do nó no fim da fila apontar para o novo nó...
    - E depois FF apontar para o novo nó.



Fila (6/9)

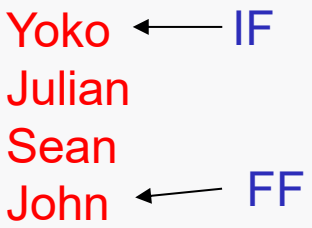
- **Inserir (enfileirar)**

```
class Fila:
…
    # fila está vazia ???
    def vazia(self):
        return self.IF is None

    # enfileirar
    def inserir(self, novo_no):
        if (self.vazia()):
            self.IF = novo_no
        else:
            self.FF.elo = novo_no

        self.FF = novo_no
        self.N += 1
        return True
```

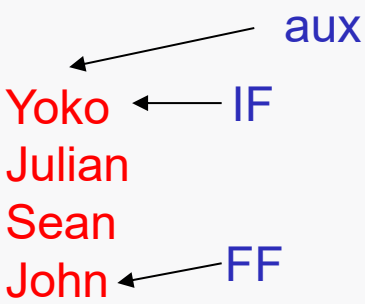
```
# consulta valor no início da fila
# (sem desenfileirar)
    def valor_inicio(self):
        return None if self.IF is None else self.IF.info

    # retorna o tamanho da fila
    def tamanho(self):
        return self.N
```

Fila (7/10)

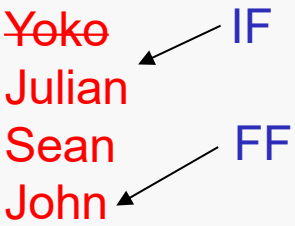
- **Operação remover (desenfileirar)**
  - Esta é a fila original “d”:



  - Variável auxiliar recebe início da fila atual.



  - Muda-se o início. Ele passa a ser o elo do início anterior.
    - E o início anterior pode ser eliminado



Fila (8/10)

- **remover (desenfileirar)**

```
class Fila:
…

    # desenfileirar
    def remover(self):
        if (self.vazia()): return None
        else:
            no_aux = self.IF
            dado = self.IF.info
            if (self.IF is self.FF): # fila tem 1 elemento
                self.IF = self.FF = None
            else: # fila tem mais de 1 elemento
                self.IF = self.IF.elo

        no_aux = None
        self.N -= 1
        return dado
```

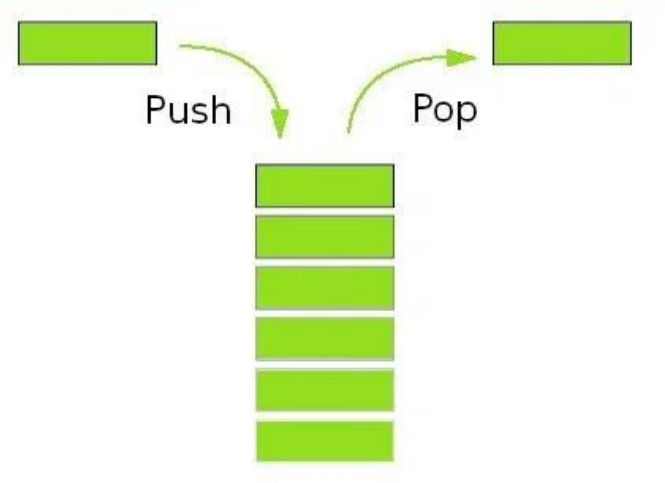
Fila (9/9)

- **Código de teste**

```
if __name__ == "__main__":
    f =Fila()
    print("* Criação da Fila:");
    print(f"N = {f.tamanho()} - Está vazia? {f.vazia()}")

    print('\nenfileirando...')
    f.inserir(No("Keith Richards")); print(f.FF.info, 'entrou na fila')
    f.inserir(No("Mick Jagger")); print(f.FF.info, 'entrou na fila')
    f.inserir(No("Charlie Watts")); print(f.FF.info, 'entrou na fila')

    print("\n* Info sobre a fila atual:")
    print("Está vazia? ", f.vazia())
    print(f"N = {f.tamanho()} - IF = {f.IF.info} - FF = {f.FF.info}")
    print('primeiro da fila:', f.valor_inicio())

    print('\ndesenfileirando...')
    for i in range(4): print(f.remover())

    print()
    print("\n* Info sobre a fila atual:")
    print(f"N = {f.tamanho()} - Está vazia? {f.vazia()}")
```

Pilha (1/10)

- **O que é Pilha (*stack*) ?**
  - É um container de elementos que são inseridos e removidos de acordo com o conceito **LIFO** (*Last-In-First-Out* = último a entrar é o primeiro a sair).
    - É como se fosse uma pilha de pratos em um restaurante. Cada prato é sempre inserido ou retirado do topo da pilha.
  - Possui apenas **duas operações** de modificação:
    - empilhar (*push*) = adicionar elemento ao topo.
    - desempilhar (*pop*) = remover elemento do topo.



Pilha (2/10)

- **Pilha – Exemplo de aplicação**
  - Funcionalidade desfazer (CTRL + Z)
    - Baseia-se na construção de uma pilha com as *n* últimas alterações.
    - Para desfazer as digitações, basta sair desempilhando... automaticamente, tudo vai ser feito na ordem certinha.

1-Digitou a frase: "*Eu amo Estatística*"

2-Deu um CTRL + Z: "*Eu amo*"

3-Digitou "pizza": "*Eu amo pizza*"

Pilha (3/10)

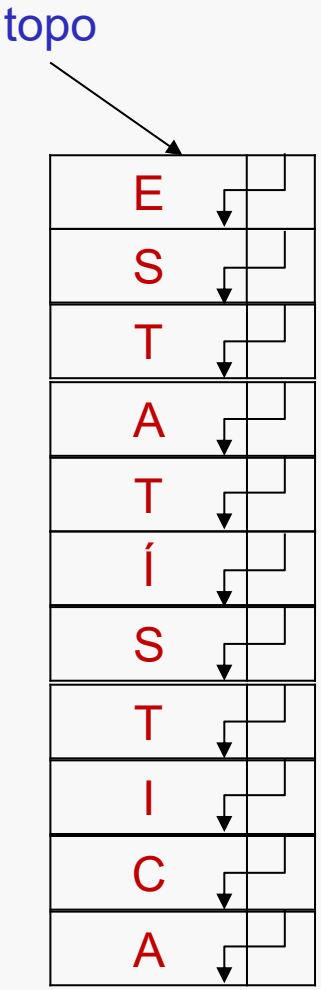
- **Vamos implementar uma Pilha em POO com os seguintes métodos:**

Nome	Complexidade	Utilização	Descrição
__init__	O(1)	p = Pilha()	Cria a pilha
empilhar	O(1)	p.empilhar(x)	Empilha o item *x*
desempilhar	O(1)	x = p.desempilhar()	Desempilha e retorna o elemento do topo.
valor_topo()	O(1)	x = p.valor_topo()	Retorna o valor do elemento do topo sem desempilhar
vazia	O(1)	p.vazia()	Retorna True se a pilha estiver vazia ou False se tiver ao menos um elemento.
tamanho	O(1)	tam = p.tamanho()	Retorna a quantidade de elementos da pilha.

- Assim como na Fila, todas as operações são O(1) !!!

Pilha (4/10)

- **Abordagem básica:**
  - A implementação será feita com base em uma lista encadeada.
  - O elemento do topo será o único diretamente acessível (pois assim é a definição de pilha).
  - Cada elemento terá um ponteiro para o elemento que estiver logo abaixo dele na pilha.
    - Esse ponteiro é chamado de **elo**.
  - *Também é possível implementar uma pilha por array, mas não veremos em nosso curso.*



Pilha (5/10)

- É preciso definir duas classes:

  - **No**: representação de um nó da fila (dados + elo)

```
class No:
    def __init__(self, dados):
        self.info = dados
        self.elo = None
```

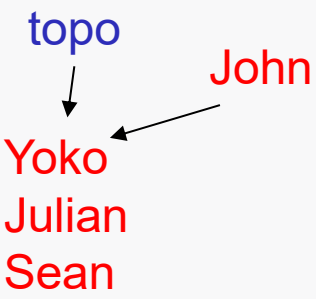
  - **Pilha**: p/ representar a pilha propriamente dita (nós empilhados)

```
class Pilha:

    def __init__(self):
        #ponteiro para o topo
        self.topo = None

        #num. de elementos
        self.N = 0
```

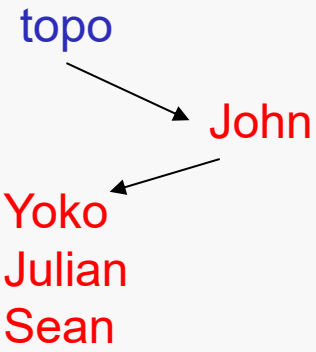
Pilha (6/10)

- **Operação empilhar**
  - Esta é a pilha original “p”:

  - Criamos um novo nó com o elo apontado para o elemento do topo:



  - Por fim, fazemos o topo apontar para novo nó



Pilha (7/10)

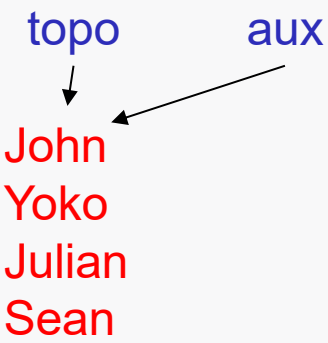
- **empilhar**

```
class Pilha:
…
    # empilhar
    def empilhar(self, novo_no):
        novo_no.elo = self.topo
        self.topo = novo_no
        self.N += 1
        return True

    # consulta valor no topo da pilha (sem desempilhar)
    def valor_topo(self):
        return None if self.topo is None else self.topo.info

    # retorna o tamanho da pilha
    def tamanho(self):
        return self.N
```

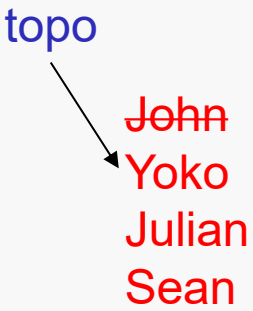
Pilha (8/10)

- **Operação desempilhar**
  - Esta é a pilha original “p”:

  - Variável auxiliar recebe topo atual.



  - Muda-se o topo. Ele passa a ser o elo do topo anterior.
    - E o topo anterior pode ser eliminado



Pilha (9/10)

- **desempilhar**

```
class Pilha:
…

    # pilha vazia?
    def vazia(self):
        return self.topo is None

    # desempilhar
    def desempilhar(self):
        if (self.vazia()): return None
        else:
            # recupera o dado do topo atual
            no_aux = self.topo
            dado = no_aux.info

            # muda o topo atual
            self.topo = self.topo.elo
            no_aux = None # "mata" o antigo topo
            self.N -= 1
            return dado # retorna o dado que estava no topo
```

Pilha (10/10)

- **Código de teste**

```
if __name__ == "__main__":
    p = Pilha()
    print("* Criação da Pilha:");
    print(f"N = {p.N} - Está vazia? {p.vazia()}")

    print('\nempilhando...')
    p.empilhar(No("Keith Richards")); print(p.valor_topo(), 'entrou na pilha')
    p.empilhar(No("Mick Jagger")); print(p.valor_topo(), 'entrou na pilha')
    p.empilhar(No("Charlie Watts")); print(p.valor_topo(), 'entrou na pilha')

    print("\n* Info sobre a pilha atual:")
    print("Está vazia? ", p.vazia())
    print(f"N = {p.tamanho()} - topo = {p.valor_topo()}")

    print('\ndesempilhando...')
    for i in range(p.tamanho()): print(p.desempilhar())

    print("\n* Info sobre a pilha atual:")
    print("Está vazia? ", p.vazia())
    print(f"N = {p.tamanho()} - topo = {p.valor_topo()}")
```

Pacote collections.deque (1/5)

- **O que é collections.deque?**
  - Significa “double-ended queue”
  - Como o nome diz, é um pacote que contém uma implementação pronta da ED **lista duplamente encadeada**.
  - Você pode acessar, remover ou inserir elementos do **início da lista** em tempo O(1).
  - E também acessar, remover ou inserir elementos do **fim da lista** em tempo O(1).
  - É uma ótima opção para implementar pilhas e filas em Python.

Pacote collections.deque (2/5)

- **collections.deque – utilizando como uma ED pilha**

```
>>> from collections import deque
>>> pilha = deque(["John","Yoko","Julian","Sean"])
>>> pilha
deque(['John', 'Yoko', 'Julian', 'Sean’])

>>> pilha.append("Paul")    # empilha (insere no final)
>>> pilha
deque(['John', 'Yoko', 'Julian', 'Sean', 'Paul’])

>>> pilha.pop()             # desempilha (remove do final)
'Paul'
>>> pilha
deque(['John', 'Yoko', 'Julian', 'Sean'])
```

Pacote collections.deque (3/5)

- **collections.deque – utilizando como uma ED fila**

```
>>> from collections import deque
>>> fila = deque(["John","Yoko","Julian","Sean"])
>>> fila
deque(['John', 'Yoko', 'Julian', 'Sean’])

>>> fila.append("Paul")    #insere no fim da fila
>>> fila
deque(['John', 'Yoko', 'Julian', 'Sean', 'Paul’])

>>> fila.popleft()         #remove do início da fila (primeiro elemento)
'John'
>>> fila
deque(['Yoko', 'Julian', 'Sean', 'Paul'])
```

Pacote collections.deque (4/5)

- **collections.deque – mais exemplos**

```
>>> from collections import deque
>>> q = deque()   # cria uma deque vazia
>>> q.append("Z")       # insere um elemento
>>> q.appendleft("X")   # insere um elemento no início
>>> q.insert(1, "Y")      # insere elemento na posição 1
>>> q
deque(['X', 'Y', 'Z’])

# testa se a estrutura contém elementos:
# retorna False se deque estiver vazia ou True se tiver ao menos 1 elemento.
>>> if (q): print("possui elementos")
possui elementos
```

Pacote collections.deque (5/5)

- **Principais métodos da deque**
  - q.**append(*x*)**: insere elemento *x* no final da deque - O(1)
  - q.**appendleft(*x*)**: insere elemento *x* no início da deque - O(1)
  - q.**pop()***:* remove o elemento do final da deque - O(1)
  - q.**popleft()***:* remove o elemento do início da deque - O(1)
  - q.**insert(*i, x*)**: insere elemento *x* na posição *i* - O(n)
  - q.**remove(*x*)**: remove o primeiro elemento que tiver o valor *x* – O(n)
  - q.**index(*x, inicio, fim*)**: obtém o índice da primeira ocorrência do elemento *x* na deque ou dentro da faixa especificada em *início* e *fim* – O(n)
  - q.**clear()**: esvazia a deque - O(1)
  - q.**copy()**: retorna uma nova lista que é um clone da original (deep copy) – O(n)

Mais informações em:
https://docs.python.org/3/library/collections.html#collections.deque

Link para Notebook no Colab

- **Fila, Pilha (implementação e pacote deque)**
  - https://colab.research.google.com/drive/12Mhm8NbeSDD0l0mMuFGkrnGDAxLYb_B5?usp=sharing

Exercício

- Crie o seguinte método para a classe ListaEncadeada, apresentada na aula 08:

  - **invertida()**: deve retornar uma string com a lista em ordem invertida.
    - Original: John → Yoko → Julian → Sean
    - Invertida: Sean → Julian → Yoko → John

    - **IMPORTANTE** → Elabore a sua solução utilizando uma pilha criada com o módulo collections.deque

Referências

- Edelweiss, N. e Galante, R. (2009). “Estruturas de Dados”. Bookman

- Python Docs (2020). Collections – Container datatypes https://docs.python.org/3/library/collections.html